Setting up and running Apache Airflow

In this train, we'll focus on setting up, running, and interacting with an Apache Airflow server.

Learning objectives

- Set up, run, and interact with an Airflow server.

Pre-requisite

Before setting up Apache Airflow with Docker, ensure you have the following prerequisites:

- Install **Docker Community Edition (CE)** with at least **4GB of memory** allocated (preferably 8GB).
- Install **Docker Compose v2.14.0** or newer.
- Check your memory allocation with:

```
docker run --rm "debian:bookworm-slim" bash -c 'numfmt --to iec $(echo $(($(getconf _PHYS_PAGES) * $(getconf PAGE_SIZE))))'
```

Fetching the Docker-Compose File

To deploy Airflow using Docker Compose, fetch the latest `docker-compose.yaml` file:

```
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/2.10.5/docker-compose.yaml'
```

What is the code doing?

- Downloads the `docker-compose.yaml` file from Apache Airflow.
- `-L` ensures redirects are followed, `-f` fails silently on errors, and `-O` saves the file with its original name.

Initializing the Environment

Before starting Airflow, initialize the environment:

```
mkdir -p ./dags ./logs ./plugins ./config
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

If using Windows or macOS, manually create an `.env` file with:

```
AIRFLOW_UID=50000
```

What is the code doing? ✖

> - Creates necessary directories for DAGs, logs, plugins, and configuration.
> - Sets the `AIRFLOW_UID` environment variable to ensure correct file ownership.

Starting Airflow

Run the following command to start all Airflow services:

```
docker compose up -d
```

Check the status of the running containers:

```
docker ps
```

```
docker compose run airflow-worker airflow info
```

What is the code doing? ✖

> - Starts the Airflow services in detached mode (`-d` runs in the background).
> - Ensures the database is initialized, and scheduler, webserver, worker, and other services start up.
> - `docker ps` Lists active Docker containers and verifies that all required services are running.
> - Ensures none of the containers are in an unhealthy state.
> - Executes an Airflow command inside the `airflow-worker` container.
> - `airflow info` provides system information and verifies the installation.

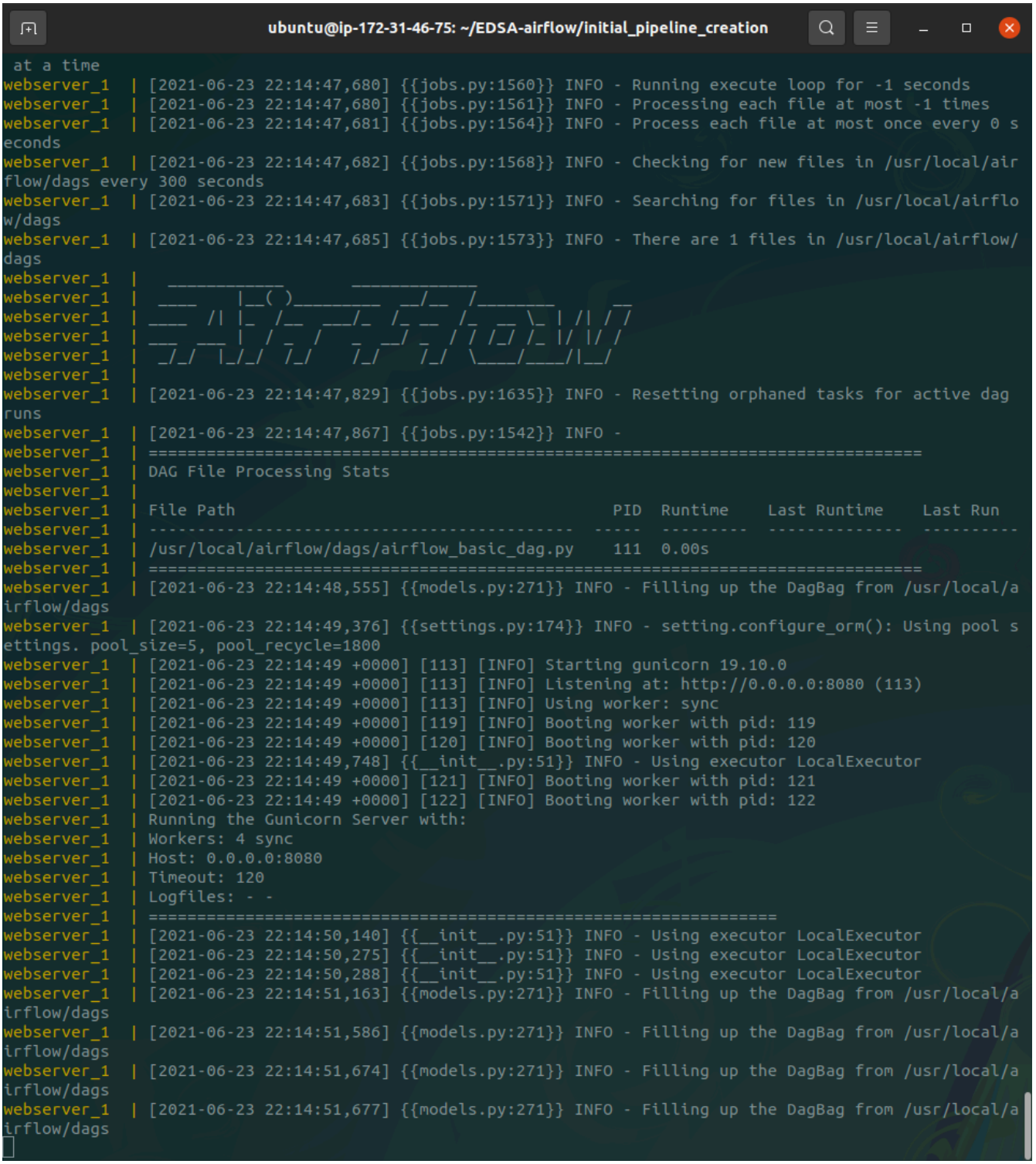


Figure 1: Once the 'docker-compose up' command is run, a large amount of logging text should appear in the terminal, indicating the Airflow docker containers are setting themselves up. After the Airflow logo is printed (as seen above), the Airflow service will be ready for operation.

Accessing Airflow

- **Web Interface:** Visit `http://localhost:8080` (default credentials: `airflow / airflow`).
- **CLI Commands:** Run CLI commands inside a container:

```
docker compose run airflow-worker airflow info
```

- **REST API:** Test the API with:
- ```
  curl -X GET --user "airflow:airflow" "http://localhost:8080/api/v1/pools"
  ```

What is the code doing? ✖

- Username: `airflow`
- Password: `airflow`
- Uses `curl` to send a GET request to the Airflow REST API.
- Provides authentication via `--user "airflow:airflow"`.

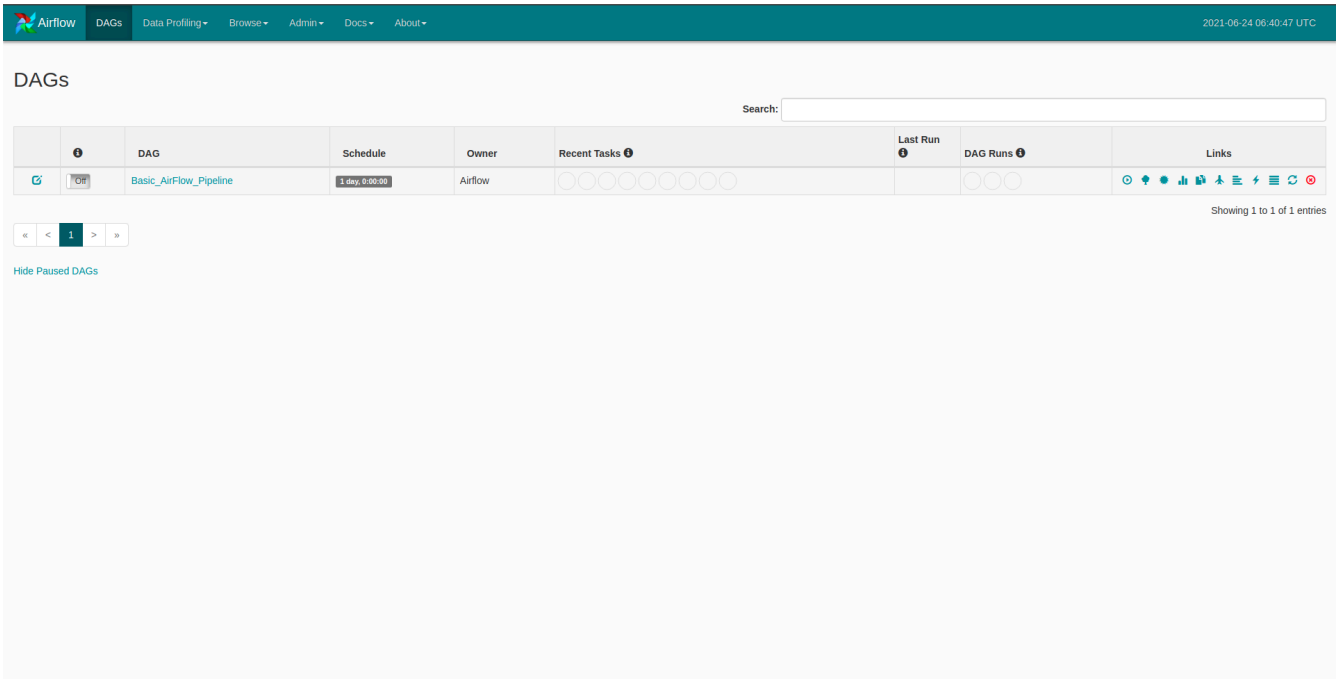


Figure 2: The Airflow UI's overview page.

Cleaning Up

To stop and remove containers, along with their volumes, run:

```
docker compose down --volumes --rmi all
```

What is the code doing? ✖

- `down` stops and removes containers.
- `--volumes` deletes associated storage volumes.
```

> - `--rmi all` removes all related images to free up space.

Conclusion

Setting up Apache Airflow with Docker ensures a quick and efficient deployment process. With Docker, managing dependencies becomes easier, and Airflow runs in an isolated environment. By following these steps, you can successfully launch and manage workflows using Airflow.

Key Takeaways ✖

> - **Takeaway 1: Docker Benefits:** Provides an isolated environment, making it easy to deploy Apache Airflow.
> - **Takeaway 2: Airflow Setup:** Requires Docker and Docker Compose for a streamlined installation.
> - **Takeaway 3: Web Interface:** Airflow UI can be accessed through `http://localhost:8080` after setup.
> - **Takeaway 4: Containerized Execution:** Airflow components run within separate containers for better management.
> - **Takeaway 5: Workflow Optimization:** Running Airflow with Docker enhances reproducibility and ease of maintenance.